Million-Parameter Python Code Generation for Microcontrollers: Signature Anchoring and Compiler-Gated Repair

Rohith Yarramala

***Abstract*—Can a code model small enough for microcontroller flash write useful embedded utility functions? We train decoder-only Transformers with 0.53 M and 1.05 M parameters on a leak-audited benchmark of IoT-oriented Python tasks and run them with a portable INT8 engine, including the unmodified ESP32-S3 firmware in host simulation; the firmware builds for the ESP32-S3 but has not yet been run on hardware. On unseen wordings of known tasks, the 0.53 M model, which needs 588,504 bytes of flash, passes the unit tests on 82.4 % of tasks with *signature anchoring* and 85.7 % with repair gated by the compiler and a static name check (mean of 3 seeds). Repair mainly fixes syntax, is no better than resampling at equal cost, and does not need the error message. On unseen task families the mean accuracy falls to at most 2.4 %: such models are verifiable specialists for a closed catalogue of device functions, not general programmers.**

***Index Terms*—TinyML, code generation, small language models, program repair, microcontrollers.**

I. Introduction

The headline code-generating language models [1], [2] have billions of parameters, while microcontrollers have megabytes of memory and are often offline [3]–[5]. Even "small" models such as MobileLLM [6] have hundreds of millions of parameters, but TinyStories [7] showed that a few million parameters suffice for fluent text in a narrow domain. We ask what a code model that fits in microcontroller flash can do, and which inexpensive decoding techniques make it more reliable. A compiler detects syntax errors for free, and feeding its diagnostics back to a model is established for large models [8]–[10]; yet at equal sampling budget, self-repair is often no better than drawing more samples [11]. We test this at a scale about 10^4 times smaller, where every extra generation costs time and energy on the device.

Contributions. (i) A benchmark of 48 training and 16 held-out embedded task families with executable tests and an automated leakage audit. (ii) Transformers of 0.53 and 1.05 M parameters with an exact flash and RAM budget, packaged for deployment through a dependency-free INT8 C engine and ESP-IDF firmware. (iii) An equal-budget comparison of signature anchoring, compiler-gated repair with and without the error message, and resampling, over three seeds. (iv) Negative results reported as such: no generalisation to unseen families or to MBPP [12].

R. Yarramala is with the Robotics and Automation Group, Defence Institute of Advanced Technology (DIAT), Pune 411025, India (e-mail: yarramala.maecas26@diat.ac.in).

Code, data, models and firmware: https://github.com/rohithyarramala/microcoder (archived: https://doi.org/10.5281/zenodo.24187363). An extended preprint is available at https://doi.org/10.5281/zenodo.24189966.

II. Related Work

Small and on-device models. TinyML deploys networks on microcontrollers through quantisation [13], specialised runtimes [4] and joint architecture–runtime design [5], with benchmarks for vision and audio [14]. For language, narrow high-quality data lets small models punch above their size [7], [15], but compact models such as TinyLlama [16] and MobileLLM [6] are still far larger than microcontroller memory, and pretrained code models such as CodeT5 [17] still have tens of millions of parameters. We are not aware of code-generation results for models that run from microcontroller flash.

Repair from compiler feedback. DeepFix [8] and Dr-Repair [9] train models to fix compiler errors, Break-It-Fix-It [18] learns to repair real broken code with a learned breaker and verified fixes, and Self-Debug [10], Self-Refine [19] and Reflexion [20] prompt large models with feedback; see [21] for program repair in general. Olausson et al. [11] showed that repair must be compared with resampling at equal cost. We follow that protocol, and we also tested learning from the model's own errors: on training prompts the model produced only 1 syntax error in 15,328 samples, so there is nothing to learn from without leaking test wordings.

III. Benchmark, Model and Decoding

Tasks and splits. A *family* is a parameterised task with four natural-language phrasings, a reference solution and a test-input generator; tasks sample function names, argument names and constants (e.g. ADC resolution, filter coefficient, bit index). The 64 families cover sensor/IoT utilities (scaling, filtering, debouncing, checksums, byte packing, bit tests), math, list, string and recursion tasks. With a fixed seed, 16 families are held out from training. The *in-family* test split (384 tasks) uses phrasing 4 of the training families, which is never trained on; the *out-of-family* split (240 tasks) uses the held-out families. After two initial runs that were evaluated on the test split (reported in the extended version), design choices were made on a separate development split (phrasing 3). A script verifies zero exact or 8-gram instruction overlap between training and the in-family test split and zero held-out families in training. A task passes if the program returns the expected value on all five test inputs in a sandbox with a 2 s time limit.

Model. A decoder-only Transformer [22] with RMSNorm [23], rotary embeddings [24], GELU MLP, no biases and tied embeddings [25], and a byte-level BPE tokenizer [26]

TABLE I
BUDGET PER GENERATED TOKEN FOR MODEL S (0.53 M) AND M (1.05 M). HOST: ONE X86 CORE, SAME C ENGINE. ESP32-S3 LATENCY IS AN ANALYTICAL ESTIMATE, NOT A MEASUREMENT.

	S	M
Flash: INT8 model + tokenizer (B)	588,504	1,118,680
RAM: state incl. KV cache (B)	626,688	1,094,656
MACs/token, position 256	675,840	1,310,720
Host decode (ms/token)	0.373	0.708
ESP32-S3 decode, estimate (ms/token)	10–24	–

with 2,048 entries. Model S has 3 layers of width 96 (529,056 parameters) and model M has 4 layers of width 128 (1,049,728 parameters). Training uses 7,664 instruction–code pairs and 39,434 synthetic repair examples (instruction, corrupted program, CPython error, correct program), with the loss on answer tokens only, for 4,000 steps in 18–33 min per model on a CPU, without a GPU.

Decoding (at most four generations per task). *Signature anchoring* (SA) extracts the first `name(args)` from the request and forces the answer to begin with `def name(args):`. *Recursive Error-Feedback* (REF) re-prompts the model with the request, the faulty draft and the error message while a gate rejects the program, drawing a sample at temperature 0.7 if greedy repair repeats the draft; the gate is `compile()`, optionally followed by a 118-line static check for names that are read but never bound, written without `re` or `ast` so that it can run in MicroPython. We compare the full CPython message, a line-only message (roughly what MicroPython reports) and a constant placeholder instead of the message, and the equal-budget baseline of [11]: resample until the gate passes.

IV. DEPLOYMENT PATH FOR THE ESP32-S3

Weights are quantised per output row to INT8 with FP32 scales; activations and the KV cache stay FP32 for the ESP32-S3's single-precision FPU. The C99 engine reads weights in place from flash, does not allocate in the decode loop and includes the tokenizer; it reproduces the reference tokenizer on all 18,524 benchmark and corpus strings, and on a test prompt its logits match PyTorch with the same quantised weights to within 10^{-4} (model M). Running the full benchmark through the INT8 engine instead of FP32 PyTorch leaves the seed-0 SA pass rate unchanged (model M: 70.8 and 70.8 %; model S: 81.5 and 81.5 %). Built with ESP-IDF v5.4.2, the firmware image with model S and all 624 test prompts is 870,176 bytes; static data-RAM (DIRAM) use is 55,871 of 341,760 bytes, and the KV cache for the 256-token context (589,824 bytes) requires a module with PSRAM. The unmodified `app_main()`, run on a PC under an emulated ESP32-S3 heap budget (300 KiB internal, 8 MiB PSRAM), passes 51.9 % of the 624 prompts with SA and greedy decoding, matching PyTorch. We also provide a MicroPython native module and a REF loop that uses MicroPython's own `compile()` as the gate; neither has yet been run on a device.

Table I gives the budget. We have not yet measured latency on ESP32-S3 hardware; the estimate assumes 4 cycles per INT8 multiply–accumulate at 240 MHz at mid-context (position 128), compute-bound for the lower value and with all weights also streamed from external flash at 40 MB/s per token for the upper value, and a board run (`esp32/run_lab.sh board` in the repository) replaces it with measured latency and on-device accuracy.

TABLE II
PASS RATE (%), MEAN ± S.D. OVER 3 SEEDS. ID: IN-FAMILY (384 TASKS); OOD: OUT-OF-FAMILY (240 TASKS), MODEL M. "GENS": MEAN GENERATIONS USED (BUDGET 4), MODEL M.

Strategy	S, ID	M, ID	M, OOD	Gens
Greedy	70.9±1.9	67.9±12.5	0.7	1.00
SA	82.4±1.3	77.5±7.1	1.2	1.00
SA + resample	–	77.9±7.1	1.4	1.12
SA + REF, full msg.	84.0±1.2	78.3±6.8	1.2	1.10
SA + REF, line only	–	78.4±6.8	1.2	1.10
SA + REF, no msg.	–	78.6±6.7	1.4	1.10
SA + REF, two gates	85.7±2.0	79.8±6.0	1.2	1.26

V. RESULTS

Signature anchoring is the largest gain (Table II). For model M it raises the in-family pass rate from 67.9 to 77.5 % at no cost: without it, the model often writes a correct body under a slightly different header, which fails every test.

Repair fixes syntax, not behaviour, and the message does not matter. REF raises syntactic validity from 95.2 to 97.7 % in-family and from 69.2 to 86.0 % out-of-family, but the pass rate barely moves. With the full message, a line-only message and no message, SA+REF reaches 78.3, 78.4 and 78.6 %; resampling reaches 77.9 %. All differences are far smaller than the seed spread, consistent with the finding of [11] that at equal cost self-repair gains little over resampling; here the content of the message makes no measurable difference. For a device, this means the cheaper line-only message of MicroPython loses nothing.

A second free gate helps a little. Of the 108 in-family failures of SA+REF (seed 0), 28 are `NameErrors` that `compile()` accepts. Gating on `compile()` plus the name check gives the best result, 79.8 % for model M with 1.26 generations on average; the check has no false positives on all 5,581 reference solutions. The gain is consistent across seeds but smaller than their spread.

Half a megabyte is enough. Model S is at least as accurate as model M (82.4 ± 1.3 % with SA and 85.7 % with SA+REF and two gates) and is more stable across seeds; a 3.05 M model, trained for 3,000 steps, reaches 69.0 % with SA (one seed). For a closed task catalogue, model size does not appear to be the bottleneck. Without the repair examples, greedy accuracy was lower in a single-seed run (45.6 vs. 59.6 %), suggesting that repair data also acts as extra supervision. Instruction augmentation, especially random identifiers, *lowered* development-split accuracy (one seed each), presumably because descriptive names are a strong cue to the intended task.

No generalisation beyond the catalogue. On held-out families every strategy passes 0.7–2.4 % of tasks (means over

seeds, both models), although REF makes most programs compile. On the 500 MBPP test problems model M (seed 0) writes valid Python for 47.8 % of them but passes 0. The model recalls and adapts known tasks; it does not compose new algorithms.

VI. Discussion and Conclusion

A model of about half a megabyte can serve as an offline, verifiable generator for a closed catalogue of device functions, such as unit conversions, filters, thresholds, checksums and bit operations, adapted by a technician in the field. At this scale the decoding procedure matters more than error feedback: signature anchoring plus at most three gated retries (1.26 generations on average) captures nearly all of the available accuracy. *Limitations:* the benchmark is synthetic, and 355 of the 384 in-family reference solutions also occur in training, so the in-family split tests recognition of known tasks from unseen wording; the seed spread is large; the gates check form, not function; and the model has not yet been run on ESP32-S3 hardware, so on-device latency and accuracy are not yet measured. Code, data, models, firmware and the scripts that generate every number in this letter are public.

Acknowledgment

AI disclosure: The author used Anthropic's Claude, through the Claude Code agent, to write and refactor parts of the software (data generation, training, evaluation, the C engine and the firmware), to run the experiments, and to draft and edit the text of all sections. All reported numbers are produced by the released scripts and inserted automatically from their result files. The author reviewed and verified the code, the results and the text, and takes full responsibility for the content.

References

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] Y. Li *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[3] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[4] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021.

[5] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[6] Z. Liu *et al.*, "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024.

[7] R. Eldan and Y. Li, "TinyStories: How small can language models be and still speak coherent English?" *arXiv preprint arXiv:2305.07759*, 2023.

[8] R. Gupta, S. Pal, A. Kanade, and S. Shevade, "DeepFix: Fixing common C language errors by deep learning," in *Proc. AAAI Conf. Artificial Intelligence*, 2017, pp. 1345–1351.

[9] M. Yasunaga and P. Liang, "Graph-based, self-supervised program repair from diagnostic feedback," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020.

[10] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[11] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[12] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[14] C. Banbury *et al.*, "MLPerf Tiny benchmark," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[15] S. Gunasekar *et al.*, "Textbooks are all you need," *arXiv preprint arXiv:2306.11644*, 2023.

[16] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.

[17] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 8696–8708.

[18] M. Yasunaga and P. Liang, "Break-it-fix-it: Unsupervised learning for program repair," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021.

[19] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[20] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[21] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[23] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[24] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[25] O. Press and L. Wolf, "Using the output embedding to improve language models," in *Proc. Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2017, pp. 157–163.

[26] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1715–1725.